

Assignment Report on GUI-Based Multi-Client Chat Application Using TCP

Submitted By	
Student Name:	Shahan Uz Zaman
Roll Number:	CS-BTC24-23
Course / Subject Name	Summer internship on Cyber Security and Secure App Development

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application while reusing the existing server implementation. The project introduces graphical user interface (GUI) programming, event-driven programming, multithreading, and user-friendly network application development using Python Tkinter. The networking logic remains unchanged, while the client interface is replaced with a graphical application.

2. GUI Design

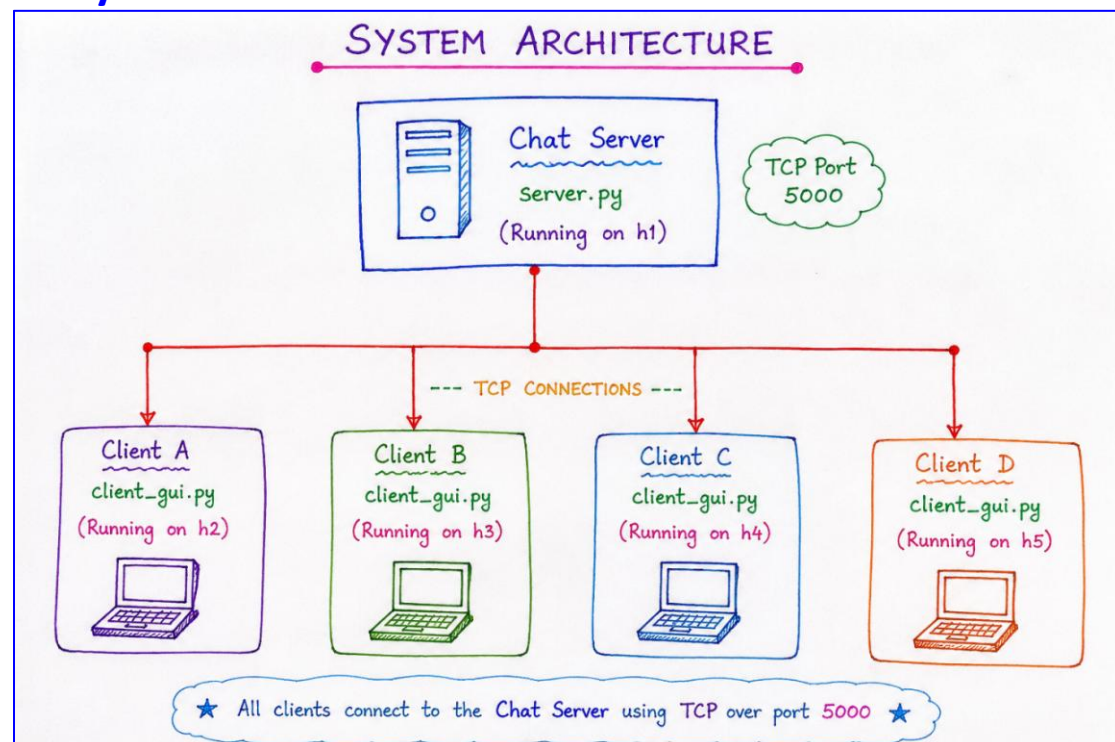
The application provides a simple desktop interface developed using Python Tkinter.

The GUI consists of:

- Login Window
- Username field
- Optional password field
- Connect button
- Chat window
- Online users panel
- Scrollable message area
- Private message field
- Message input box
- Send button
- Disconnect button
- Connection status indicator

The GUI is event-driven and remains responsive because incoming messages are received using a background thread.

3. System Architecture



Working Principle

- The server starts and listens on TCP port 5000.
- Clients connect to the server using TCP sockets.
- Users enter a username through the GUI.
- The server authenticates the username.
- Messages are exchanged through TCP sockets.
- The server broadcasts public messages.
- Private messages are delivered only to the selected client.
- Chat history and performance statistics are maintained.

4. Components Reused from Assignment 5

The Assignment 5 server implementation was reused with minimal modifications.

The following components were reused:

- TCP socket communication
- Multi-client server
- Username management
- Broadcast messaging
- Private messaging
- Online user management
- Chat history logging
- Performance statistics
- CSV logging
- Background client threads

5. GUI Design Decisions

The following design decisions were taken during implementation:

- Tkinter was selected because it is included with Python.
- Networking logic and GUI logic were kept separate.
- A background thread receives incoming messages.
- Scrollable chat area improves readability.
- Online user list provides information about active users.
- Double-clicking a username enables private messaging.
- Status label indicates current connection state.

6. Software Requirements

Operating System

- Ubuntu Linux

Programming Language

- Python 3

Python Libraries

- socket
- threading
- tkinter
- tkinter.ttk
- tkinter.scrolledtext

Network Emulator

- Mininet

Packet Analyzer

- Wireshark

7. Network Topology

The assignment was tested using Mininet.

sudo mn --topo single,5

Network configuration:

Host	Purpose
h1	Chat Server
h2	Client A
h3	Client B
h4	Client C
h5	Client D

All hosts successfully communicated over the virtual network.

8. Implementation

Server Features

- TCP socket server
- Multiple client handling
- Broadcast messaging
- Private messaging
- Online user tracking
- Chat history logging
- Performance statistics
- Graceful disconnection
- Client Features
- Graphical login window

Chat interface

- Scrollable chat area
- Online user list
- Private messaging
- Send button
- Disconnect button
- Connection status
- Background receive thread

9. Testing Results

Several test cases were performed to verify the application.

Test Case	Expected Result	Status
Server starts	Server listens on port 5000	Pass
Client login	Successful connection	Pass
Broadcast message	Message received by all clients	Pass
Private message	Delivered only to selected client	Pass
Online users	User list updated	Pass
Disconnect	User removed from online list	Pass
Multiple clients	Simultaneous communication	Pass

10. Wireshark Verification

Wireshark was used to capture TCP packets exchanged between the server and clients.

Display filter used:

tcp.port == 5000

The following events were verified:

- Client connection
- Broadcast message
- Private message
- Client disconnection

11. Reflection Answers

A. Why should networking logic and GUI code be separated?

Separating networking logic from the GUI improves code organization, readability, and maintainability. The networking functions can be reused independently of the graphical interface, making future development and debugging easier.

B. Why is a background thread required?

Socket operations such as `recv()` are blocking operations. Executing them in the main GUI thread would freeze the application while waiting for incoming messages. A background thread allows the GUI to remain responsive while messages are received continuously.

C. Which components from Assignment 5 were reused?

The following components were reused:

- TCP socket communication
- Multi-client server
- Broadcast messaging
- Private messaging
- Online user management
- Chat history
- CSV logging
- Performance statistics

Only the client interface was changed from terminal-based to graphical.

D. How did the GUI improve usability?

The graphical interface is easier to use than the terminal interface. Users can send messages using buttons, view online users, select recipients for private messages, and read chat history in a scrollable window without memorizing terminal commands.

E. Suggest one future enhancement.

Future enhancements may include:

- File sharing
- Emoji support
- Image transfer
- Voice communication
- Group chat rooms
- User authentication with a database
- Message encryption
- Chat notifications

12. Conclusion

This assignment successfully transformed the terminal-based TCP chat client into a graphical desktop application while reusing the networking logic developed in Assignment 5. The application supports multiple clients, broadcast messaging, private messaging, online user management, and responsive communication through multithreading. Testing in the Mininet environment confirmed reliable communication among clients, and Wireshark verified the TCP packet exchange. The project demonstrates the integration of networking concepts with GUI programming and provides a user-friendly interface for TCP-based communication.